

Tratamento de exceções

O desenvolvimento de programas, mesmo quando são utilizados algoritmos eficientes e bem estruturados, pode enfrentar situações que fogem ao controle da pessoa desenvolvedora. Erros decorrentes de entradas de dados inadequadas ou indisponibilidade de serviços são comuns durante a execução de programas, podendo causar interrupções e impactar negativamente a experiência do usuário. Para lidar com esses desafios, as linguagens de programação oferecem mecanismos de tratamento de exceções, que permitem identificar essas inconsistências e aplicar soluções que garantam a estabilidade do software (Guilhon *et al.*, 2022; Lambert, 2022).

Nesse contexto, o primeiro subcapítulo aborda os conceitos e aplicações do tratamento de exceções na linguagem Python, explorando diferentes cenários de uso. O segundo subcapítulo discute como a pessoa desenvolvedora pode definir seus próprios critérios de tratamento de erros, disparando as exceções de maneira personalizada.

1 Conceito e aplicações

Quando nos referimos ao tratamento de exceção em linguagens de programação, é importante ressaltar que exceções e erros de sintaxe são situações diferentes. O erro de sintaxe está relacionado à escrita do código-fonte de maneira incompatível com o interpretador ou compilador da linguagem de programação alvo. Já uma exceção acontece mesmo com o código-fonte escrito corretamente, o que depende de situações que ocorrem em tempo de execução e que podem ser tratadas amigavelmente de maneira a não interromper o fluxo de execução de um programa. A sintaxe básica para o tratamento de exceção é:

```
try:
    # Código que pode gerar uma exceção
except ExceptionType:
    # Código a ser executado se a exceção ocorrer
```

Em que a palavra reservada *try* circunda o bloco de instruções que deve ser observado quanto às exceções que podem ser geradas, e *except* é o tipo de exceção que deve ser capturado no trecho circundado pelo bloco *try*. Vamos ao exemplo 1.

Exemplo 1

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("Erro: divisão por zero não permitida!")
```

O exemplo 1 provoca um erro a partir de uma operação de divisão por zero não permitida. Essa instrução, sem o devido tratamento, geraria uma interrupção na aplicação informando uma exceção de divisão por zero (*ZeroDivisionError*). No entanto, com a utilização da estrutura de tratamento de exceção, o programa, em vez de gerar o erro e encerrar, exibe a mensagem "Erro: divisão por zero não permitida!", contemplada no bloco *except*.

Além da captura de uma única exceção, podem ser capturadas mais exceções realizando o tratamento em específico por cada bloco `except`, como no exemplo 2 a seguir, no qual há a captura de erros do tipo `ZeroDivisionError` ou `ValueError` (por exemplo, se o usuário insere um caractere não identificado por um número) em blocos de instrução separados.

Exemplo 2

```
try:
    num = int(input("Digite um número: "))
    resultado = 10 / num
except ValueError:
    print("Erro: você deve digitar um número inteiro válido.")
except ZeroDivisionError:
    print("Erro: divisão por zero não é permitida.")
else:
    print(f"Resultado: {resultado}")
```

Além do uso dos blocos `try` e `except`, os blocos `else` e `finally` também podem ser utilizados nas seguintes situações: o bloco `else` é executado caso nenhuma exceção seja capturada no bloco `try`, e o bloco `finally` sempre é executado, independentemente de ser capturada uma exceção ou não, com mostra o exemplo 3.

Exemplo 3

```
try:
    arquivo = open("dados.txt", "r")
    conteudo = arquivo.read()
except FileNotFoundError:
    print("Arquivo não encontrado.")
else:
    print("Arquivo lido com sucesso!")
finally:
    print("Encerrando o processo.")
```

O código tenta abrir o arquivo `"dados.txt"` em modo de leitura (`"r"`) dentro do bloco `try`. Se o arquivo não for encontrado, a exceção

`FileNotFoundError` será capturada pelo bloco `except`, exibindo a mensagem “Arquivo não encontrado”. Caso o arquivo seja encontrado e lido com sucesso, a mensagem “Arquivo lido com sucesso!” será exibida no bloco `else`. Independentemente de ocorrer erro ou não, o bloco `finally` sempre será executado, imprimindo “Encerrando o processo”, garantindo que qualquer ação final necessária seja realizada. O quadro 1 mostra os diferentes tipos de exceção que podem ser capturadas na linguagem Python.

Quadro 1 – Tipos de exceções mais comuns em Python

TIPO DE EXCEÇÃO	DESCRIÇÃO
<code>Exception</code>	Classe base para todas as exceções em Python.
<code>ArithmeticError</code>	Classe base para erros matemáticos.
<code>ZeroDivisionError</code>	Erro ao tentar dividir um número por zero.
<code>OverflowError</code>	Erro quando uma operação aritmética ultrapassa o limite representável.
<code>ValueError</code>	Erro quando um argumento tem um valor inadequado para uma operação.
<code>TypeError</code>	Erro quando uma operação ou função é aplicada a um objeto de tipo inadequado.
<code>IndexError</code>	Erro quando um índice de uma sequência está fora do intervalo válido.
<code>KeyError</code>	Erro quando uma chave não é encontrada em um dicionário.
<code>FileNotFoundError</code>	Erro ao tentar abrir um arquivo que não existe.
<code>IOError</code>	Erro ao lidar com operações de entrada e saída.
<code>ImportError</code>	Erro ao tentar importar um módulo que não pode ser encontrado.
<code>ModuleNotFoundError</code>	Erro quando um módulo específico não pode ser encontrado.
<code>AttributeError</code>	Erro ao tentar acessar um atributo inexistente de um objeto.
<code>NameError</code>	Erro quando uma variável ou função não foi definida antes do uso.
<code>IndentationError</code>	Erro quando a indentação do código está incorreta.
<code>SyntaxError</code>	Erro de sintaxe no código fonte do Python.

TIPO DE EXCEÇÃO	DESCRIÇÃO
RuntimeError	Erro genérico que ocorre durante a execução do programa.
MemoryError	Erro quando o sistema fica sem memória suficiente para uma operação.
AssertionError	Erro quando uma asserção assert falha.
KeyboardInterrupt	Erro gerado quando o usuário interrompe a execução do programa (Ctrl+C).

2 Invocando exceções manualmente

Além das exceções geradas nativamente em tempo de execução, as mesmas exceções podem ser geradas manualmente em situações não mapeadas através da palavra reservada `raise`, como apresentado no exemplo 4 a seguir.

Exemplo 4

```
def verificar_idade(idade):  
    if idade < 18:  
        raise ValueError("Idade deve ser maior ou igual a  
18.")  
    return "Acesso permitido."  
  
try:  
    print(verificar_idade(16))  
except ValueError as e:  
    print(f"Erro: {e}")
```

O código define a função `verificar_idade(idade)`, que verifica se a idade fornecida é menor que 18; caso seja, a função dispara uma exceção `ValueError` com a mensagem "Idade deve ser maior ou igual a 18". No bloco `try`, a função é chamada com o valor 16, o que aciona a exceção. O bloco `except` captura essa exceção e imprime a mensagem de erro formatada com o texto "Erro: Idade deve ser maior ou igual a 18", evitando que o programa seja interrompido abruptamente.

Para um uso eficaz de exceções em Python, é fundamental evitar capturas genéricas com `except Exception`, pois isso pode mascarar erros inesperados e dificultar a depuração, tornando o código menos previsível. Em vez disso, é recomendável capturar exceções específicas para lidar com cada situação de maneira apropriada. Além disso, fornecer mensagens de erro claras e informativas é essencial para facilitar a identificação e correção do problema, indicando exatamente o que aconteceu e, se possível, sugerindo uma solução.



PARA SABER MAIS

Em Python é possível personalizar a geração de exceções! Para elaborar classes personalizadas de exceção em Python, é necessário criar uma classe que herde da classe base `Exception`, permitindo definir erros específicos para um contexto particular da aplicação. Isso é útil quando os erros padrão não representam adequadamente uma situação ou quando se deseja adicionar lógica personalizada ao tratamento de exceções. Dentro da classe, pode-se sobrescrever o método de inicialização para aceitar mensagens de erro personalizadas ou outros parâmetros relevantes. Ao utilizá-la, a exceção pode ser lançada com uma mensagem clara e capturada de forma específica, melhorando a legibilidade do código, facilitando a depuração e permitindo um tratamento mais preciso dos erros.

Outra boa prática é utilizar logs em vez de `print()`, pois a biblioteca logging permite armazenar erros com diferentes níveis de gravidade (DEBUG, INFO, WARNING, ERROR, CRITICAL), além de possibilitar o envio dessas informações para arquivos ou sistemas externos, garantindo um melhor monitoramento e rastreamento de falhas no código. O código a seguir (exemplo 5) apresenta um exemplo do uso de logs em Python.

Exemplo 5

```
import logging

logging.basicConfig(level=logging.ERROR)

try:

    x = 1 / 0
except ZeroDivisionError as e:
    logging.error(f"Erro detectado: {e}")
```

O código utiliza a biblioteca logging para registrar erros em vez de simplesmente imprimir mensagens na tela. Primeiro, o módulo logging é importado, e a configuração logging.basicConfig(level=logging.ERROR) define que apenas mensagens de nível ERROR ou superior serão registradas. No bloco try, o programa tenta dividir 1 por 0, o que gera uma exceção ZeroDivisionError. O bloco except captura essa exceção e usa logging.error(f"Erro detectado: {e}") para registrar a mensagem de erro, em que {e} contém a descrição da exceção ("division by zero"). Isso permite que o erro seja tratado de maneira estruturada e registrado em logs, facilitando a depuração sem interromper a execução do programa.



IMPORTANTE

Os logs e a observabilidade são fundamentais para a manutenção e o monitoramento de servidores, permitindo detectar falhas, analisar o desempenho e garantir a segurança das aplicações. Enquanto os logs registram eventos detalhados sobre o funcionamento do sistema, incluindo erros, requisições e comportamento dos serviços, a observabilidade vai além, fornecendo uma visão holística do ambiente por meio de métricas, rastreamento distribuído e alertas proativos. O uso adequado de ferramentas de logging e monitoramento, como ELK Stack, Prometheus e Grafana, possibilita identificar gargalos, prever problemas e responder rapidamente a incidentes, garantindo maior estabilidade, eficiência e confiabilidade no ambiente de produção.

Considerações finais

É fundamental reforçar a importância de lidar com erros de forma eficiente para manter a estabilidade e previsibilidade das aplicações. Usar try-except corretamente, criar exceções personalizadas e seguir boas práticas, como evitar capturas genéricas e fornecer mensagens claras, tornam a depuração mais fácil e melhoram a experiência do usuário. Além disso, logs estruturados ajudam a monitorar o comportamento do sistema em tempo real, facilitando a detecção de problemas. Com essas estratégias, o software se torna mais robusto, confiável e preparado para enfrentar falhas de forma controlada.

Referências

GUILHON, André *et al.* (org.). **Jornada Python**: uma jornada imersiva na aplicabilidade de uma das mais poderosas linguagens de programação do mundo. Rio de Janeiro, RJ: Brasport, 2022. *E-book*.

LAMBERT, Kenneth A. **Fundamentos de Python**: estruturas de dados. Cengage Learning Brasil, 2022. *E-book*.